StratumRO Rebuild Blueprint: engine, setup sheet, and an agent workflow that does not get lost

Prepared 21 October 2026. Companion to the "StratumRO Multimodal Architecture Report" (which models to use) and the "StratumRO Execution Plan Report" (which phases and gates). This one answers how to restructure the repository, the agent workflow and the product shape so that you and your AI agents stop losing the thread.

Evidence labels: **[REPO]** read or counted in your repository, **[PUB]** from a cited public source, **[CALC]** calculated from counted numbers, **[EST]** my recommendation or estimate, **[UNVERIFIED]** I could not confirm it.

Engine, setup sheet and clients

1. Short answers

Rework, or start over? Do not start over and do not rewrite in another language. Restructure in place. The repo already has a working spine (52 root modules, 11,630 lines, plus 80 test files with 536 test functions) that produced every measurement in this project **[REPO]**. The problem is not that the engine is bad. The problem is that around it sit a second layer of unused architecture, 221 one-off scripts, 1,868 evidence files and 18 agent definitions, so neither you nor an agent can tell what is current.

App or engine? Engine first, apps as thin clients. Your own product-identity document, confirmed on 17 October, already says this: the core stays separate from the interface, QGIS is connected through verified adapters, a desktop app comes after a consolidated cadastral flow, and a Rust component needs a measured bottleneck first **[REPO]**. I agree with that decision. I read "a motor with a setup, like a car" as an engine plus a tuning sheet, so the plan below adds one setup file per run (section 6). If you meant something else by "setup car", tell me and I will adjust.

Why did you and the agents get lost? Four measurable reasons (section 2): the control layer was built before the perception core passed an evaluation, the same decisions are stored in many documents, scripts accumulated without a promotion rule, and the agent roster has more roles than one person can supervise.

2. What is in the repo today (diagnosis)

Scope of this reading: I cloned the repository at HEAD 64a5cec (20 October 2026). The clone is shallow with a single visible commit, so I could not measure history, branches or agent churn **[REPO]**. I ran static counts and read the main documents. I did not run the test suite or open QGIS **[UNVERIFIED]**.

Repository composition

2.1 Where the sprawl is

Observation	Number	Why it makes agents lose the thread
Files tracked in git	3,231	An agent that lists the repo sees mostly non-code
Of which under `workspace/`	1,868 files, 315.6 MB (1,164 jpg, 317 png, 154 json, 64 md)	Evidence images and run outputs sit beside source code
Archive folder `junk/` still tracked	424 files	Old rules and plans stay searchable and look current
Documents in `docs/`	126 files; 10 named handoffs, 10 audit reports, 28 dated 2026-10-1x	The same question has several answers
Scripts in `tools/`	221 files, 40,453 lines; 37 are `lot3` variants; the largest is 2,274 lines	One-off experiments look like product features
Scripts unrelated to the cadastral metric	A 1,016-line flight-mission simulator page builder, two drone-flight simulators, several viewer builders	Effort spent away from the 6.15% recall problem
Agent definitions in `.agents/agents/`	18 files, 49 KB; several near-duplicate pairs (`architect` and `stratumro-architect`, `reviewer` and `stratumro-reviewer`, `researcher` and `stratumro-researcher`)	Roles overlap, so it is unclear who owns a decision

All figures **[REPO]**.

2.2 The architecture that was designed but not adopted

- The 20 September architecture synthesis defines a three-level target (core, domain packs, clients) and the principle that cleanup must not change scientific behaviour **[REPO]** (`docs/architecture/STRATUMRO_FINAL_ARCHITECTURE_SYNTHESIS.md`).
- The folders exist: `core` (14 files, 1,055 lines), `domain` (18 files, 2,421 lines), `contracts` (11 files, 748 lines) **[REPO]**.
- My static scan found no root module that imports from `core`, two root modules that import from `domain`, and only 8 of 221 scripts that import `core`, `domain` or `contracts` **[REPO]**. The system map says the same in words: "migration incomplete", with two definitions of the cadastral product (a 9-line alias at the root and a 173-line class in the domain package) **[REPO]**.
- Working code is therefore in the root modules and scripts, while the layered structure is mostly a parallel copy.

2.3 The control layer got ahead of the core

- `stratum_ro/ai` has 36 files and 7,012 lines: providers (seven files), a task graph, an event bus, a router, memory, tools and a verifier. There is also a 477-line FastAPI app, a 450-line server exposing 32 tools, a 1,300-line QGIS dock widget and a web cockpit **[REPO]**.
- The measured result of the core pipeline on Cluj is a recall of 6.15% (4 of 65 references) against legal-parcel references, with a precision of 15.38% after integration **[REPO]**.
- The verifier path has a known defect: missing fields default to approved with 0.90 confidence **[REPO]**.
- The latest commit adds WebGPU inspection methods **[REPO]**. That is a UI feature, not progress on recall.

Anthropic's engineering guidance on agents says to find "the simplest solution possible", to increase complexity "only when needed", and that agent autonomy brings higher cost and compounding errors, so it should be tested in sandboxes with guardrails (Anthropic) **[PUB]**. Your project has several layers of orchestration that were not yet shown to improve the one number that matters.

2.4 What is already good (keep it)

- A single shared rule file, `AGENTS.md` (9,678 characters, 99 lines), with `CLAUDE.md`, `GEMINI.md` and `.kilorules` pointing to it instead of copying it **[REPO]**.
- Evidence rules in `.agents/rules/` (evidence, geometry, QGIS runtime, repository hygiene) **[REPO]**.
- A deterministic router that forbids the LLM from doing geometry, CRS and topology **[REPO]**.
- An evaluation-gates document and a preflight module that is among the most-imported local modules in the scripts (13 of them) **[REPO]**.
- A golden-freeze test (`test_phase1_golden_freeze.py`) and a stated invariant that cleanup must not change coordinates **[REPO]**.
- Honest status language ("pending owner review", "not measured") across the documents **[REPO]**.

3. Target shape: engine, setup sheet, thin clients

3.1 The pieces

Piece	Meaning	In your repo today
Engine	Headless Python package: stages, contracts, geodesy core. No UI imports.	`stratum_ro` root modules plus `core`; not yet cleanly separated
Setup sheet	One file per run: hardware profile, area, CRS, model preset, policies, legal profile, thresholds	Partly in code constants and docs (for example the nDSM threshold, the eave setback)
Clients	CLI, QGIS plugin, a small MCP tool set for agents, later a desktop app	Dock widget, FastAPI, 32 MCP tools, web cockpit

Piece	Meaning	In your repo today
Human gate	The authorized surveyor accepts or rejects	Already a rule
Build-time workflow	How you and agents change the code	Scattered across `.agents`, handoffs, docs

3.2 The six engine stages

1. **Ingest:** CRS check, tiling, NoData masks, source versus processing resolution recorded.
2. **Evidence:** nDSM, vegetation index, class 6 returns, as deterministic rasters.
3. **Propose:** the trained roof segmenter (L2a) and SAM (L2b), as set out in the architecture report.
4. **Geometry:** fusion, regularization, topology and area sanity, all deterministic.
5. **Judge:** embedding probe and VLM audit, advisory only, never counted as recall.
6. **Export:** GeoPackage, DXF, `.cp` coordinates, provenance.

Each stage reads and writes typed objects defined in `contracts`, and each stage can run alone on stored inputs. That makes every stage testable without a model, a GUI or a network.

3.3 Why engine-first beats the alternatives

Option	Fit with your situation	Verdict
A. QGIS plugin only	Matches what surveyors already use, but the logic gets trapped in PyQGIS and cannot run headless or on the workstation overnight	Not enough alone
B. Engine plus CLI, QGIS plugin and a small MCP set (recommended)	Same code serves you, the plugin and agents; training and batch runs happen headless on the workstation; QGIS stays the viewer and review tool	Recommended
C. Standalone desktop app now	Matches the stated long-term direction, but adds packaging, GUI and QGIS-embedding work before recall has improved; your identity document already says do the cadastral flow first	Later, after Gate A
D. Rust rewrite	Your own document requires a measured bottleneck first; none is documented	Not now

Option C stays on the roadmap exactly as you wrote it. The change is only ordering: engine and contracts first, so a future app has something clean to call.

4. Fix the agent workflow (this is where you got lost)

4.1 Separate two kinds of orchestration

Your repo mixes (a) the AI orchestration that helps build the code (Antigravity, OpenCode, Claude working on files) with (b) AI orchestration inside the product (task graphs, providers, memory in `stratum_ro/ai`). They need different rules.

- Build-time orchestration is files plus git: a task card, a branch, a review, a verify command.

- Run-time orchestration inside the product should shrink to what the router already enforces: LLM plans and explains, deterministic code decides.

4.2 Make the rules readable by every tool

- OpenCode reads `AGENTS.md` from the project root first and falls back to `CLAUDE.md` only if no `AGENTS.md` exists; extra files are added through the `instructions` list in `opencode.json`, and it does not follow file references written inside `AGENTS.md` (OpenCode docs) **[PUB]**. Your repo has no `opencode.json` **[REPO]**, so OpenCode currently sees `AGENTS.md` only. Add an `opencode.json` that lists `docs/STATE.md` (the new single status file, section 5.2).
- Antigravity discovers `AGENTS.md`, `GEMINI.md` and `.agents/rules/*.md` and combines them; rule files in `rules/` need YAML frontmatter with a trigger; the older docs cap a rule file at 12,000 characters (Antigravity rules, older rules page) **[PUB]**. Your `AGENTS.md` is 9,678 characters, which is inside that limit **[REPO]**. Check that the four files in `.agents/rules/` have frontmatter, otherwise they may be ignored **[UNVERIFIED]**.
- The Antigravity CLI documentation places custom agents at `.agents/agents/<name>/agent.md` (Antigravity CLI) **[PUB]**. Your repo stores them as flat files such as `.agents/agents/stratumro-orchestrator.md` **[REPO]**. Open the agent panel and confirm whether any of the 18 are discovered **[UNVERIFIED]**. If they are not, they are costing you reading time without helping any agent.

4.3 Shrink 18 roles to 4

Keep as agent	Replaces	Owns
Planner	architect, stratumro-architect, stratumro-orchestrator	Reads `STATE.md`, writes the task card, never edits code
Builder	stratumro-coder, gis-engineer, stratumro-geospatial, stratumro-qgis, stratumro-cad-builder	Edits only the files named in the task card
Reviewer	reviewer, stratumro-reviewer, validator, tester	Different model from the builder; reads the diff and runs `make verify`; cannot edit
Researcher	researcher, stratumro-researcher, vision-engineer	Produces a sourced note only; no code

Turn the domain specialists (`romanian-urbanism-pug-specialist`, `romanian-opendata-specialist`, `geodesy-engineer`) into skills (`.agents/skills/<name>/SKILL.md`) that any role loads when the task needs them. You already have five skills there **[REPO]**. A skill is knowledge, an agent is a role, and mixing the two is what multiplied the roster **[EST]**.

4.4 The task card (the one document an agent must read and write)

Create a `tasks/` folder. One file per task, 40 lines at most:

```
# T-0xx  Short title
Goal:           one sentence, observable
Why:            which number or gate it moves (for example: object recall on the fr
Files allowed:  exact paths; everything else is read-only
```

```
Do not touch:    accepted outlines, ground truth, other agents' branches
Acceptance:      the command to run and the result expected
Stop rules:      stop and write the handoff if tests fail twice, or if scope grows be
Status:          open | building | review | merged | rejected
Handoff:         appended here when finished (what changed, evidence, risks, next ste
```

Rules that follow from it:

1. No task without a number or gate it moves. If the "Why" line is empty, the task is a deviation (the same test as in the execution plan).
2. One writer per branch; use a separate git worktree for each agent so two tools never edit the same working copy **[EST]**.
3. The builder never reviews its own work.
4. A merge needs the verify command to pass (`python scripts/validate_project.py --mode fast`, `python -m unittest discover stratum_ro/test`, `python tools/verify_ground_truth_and_metrics.py`, as your repo already requires) **[REPO]**.
5. Handoffs are appended to the task card. No new top-level handoff documents.

4.5 One status file

`docs/PROJECT_STATE.md` is 20,348 characters and `docs/AGENT_HANDOFF.md` is 8,116 **[REPO]**. Replace them with a short `docs/STATE.md` that fits in one screen: current phase, the one headline metric on the frozen area, the open task cards, and the three decisions that bind everyone. Keep the long files as archive. An agent should be able to read `AGENTS.md` and `STATE.md` and know what to do in under two minutes **[EST]**.

5. Rework in order

Each step is one or more task cards. Nothing is deleted without your approval; moving to an archive is reversible.

R0. Safety net

- Tag the current state (`pre-restructure-2026-10`) and take a full backup of the repo and the data folders **[EST]**.
- On your machine, run the three verify commands and save the output as the baseline. The September documents report 205 passed and 9 skipped, and the migration plan lists other baselines, so the true current number is **[UNVERIFIED]** until you run it.

R1. Make the repository navigable

1. Add a root `README.md` section "Where things are" with six lines: engine, tools, tests, docs, data, archive.
2. Move evidence out of the code tree. Stop tracking `workspace/` images and run outputs (`git rm --cached`, then ignore), store them in a data folder with a manifest of hashes **[EST]**. This

does not rewrite history; rewriting history is a separate decision that I would not make without your approval.

3. Archive old plans. `junk/` already exists as an archive; add `docs/archive/` and move the older handoffs and audit reports there, leaving an index of what they contain.
4. Keep one live architecture page (`SYSTEM_MAP.md`) and the Mermaid diagrams already in it.

R2. Rebuild the working agreements

1. Reduce the agent roster to the four roles (4.3).
2. Add `opencode.json`, `tasks/`, `docs/STATE.md` (4.2, 4.4, 4.5).
3. Add a `make verify` target (or one script) that runs the three commands.
4. Confirm discovery in each tool: the agent list in Antigravity, and the loaded instructions in OpenCode **[UNVERIFIED]**.

R3. Create the setup sheet and the headless entry point

1. Write the setup file schema (section 6) and one real example for the Cluj frozen area.
2. Add a single entry command that reads a setup sheet and runs stages 1 to 6 on stored data, for example `python -m stratum_ro run --setup setups/cluj_frozen.toml` (the command name is a proposal) **[EST]**.
3. Make the entry command call the existing root modules as they are. Do not move code yet. The result must reproduce the current pipeline's outputs, including the 4 of 65, 15.38% and 69.5% figures, within rounding **[REPO]**.

R4. Promote, archive, or delete scripts

Classify the 221 scripts by the rule "does an engine stage call it, or is it a one-off":

Class	Rule	Action
Promote	Used by the pipeline or by verification (for example the `run_*`, `verify_*` families, the evaluation preflight users)	Become sub-commands of the entry point
Archive	A one-off for a lot or an investigation (the 37 `lot3` scripts, `phase3_experiment_runner`, repair scripts)	Move to `tools/archive/` with a README line
Extras	Viewers, simulators, catalogs (flight-mission simulator, dual viewers, WebGPU methods)	Move to `extras/`; not part of the product until Gate A
Delete candidates	No importers and no recent use	List for your approval; do not delete automatically

R5. Consolidate the layers, strangler style

Use the strangler pattern: wrap, switch callers, then remove, one module at a time **[EST]**.

1. Pick the contract types for the six stages (typed dataclasses in `contracts`).

2. For each stage, put a thin adapter in `core` or `domain` that calls the existing root module and returns the contract type. Add a golden test that compares old and new outputs.
3. Switch callers (entry point, QGIS plugin, MCP tools) to the adapter.
4. Only then move the code behind it. Do not move `vectorizer.py`, `lidar_processor.py`, `cad_exporter.py` or `sam2_engine.py` until their golden tests exist; the migration plan already names them as authoritative engines that must not be refactored casually **[REPO]**.
5. Remove the duplicate cadastral product definition once all callers use one.

R6. Cut the runtime AI layer to what is used

- Keep the router and its rule that geometry, CRS, topology, CAD export and nDSM are never given to an LLM **[REPO]**.
- Reduce the 32 MCP tools to about eight that matter: describe project, list data, run a stage, read an evidence packet, run verify, read provenance, list candidates, record a verdict **[EST]**.
- Reduce providers from seven to the few you actually use: one hosted model from each family you subscribe to, and one local OpenAI-compatible endpoint for the private fallback **[EST]**. The 20 September synthesis already marks Ollama as excluded, so decide that explicitly rather than keeping a dead provider.
- Keep the VLM verifier only as an audit with the defects fixed, as in the architecture report.

R7. Then continue the product phases

Only after R0 to R6 do the execution plan's phases run on a clean base: the frozen area, the roof segmenter A/B, the PUG base, Gate A, then SNAP and the CAD adapters. R1 to R6 are preparatory and should not take over the project; each step has a pass/fail check in section 8.

6. The setup sheet (proposed schema)

This is a proposal, not an existing file. Values marked with `?` are for you to fill in. Everything else comes from facts in your repo or from the earlier reports.

```
# setups/cluj_frozen.toml  (example, one file per run)
[run]
name      = "cluj_frozen_baseline"
phase     = "cadastre"            # cadastre | pug_base | snap | cad_export
hardware  = "laptop"              # profile defined below

[area]
aoi       = "data/aoi/cluj_frozen_test.gpkg"
crs       = "EPSG:3844"
source_gsd_m     = 0.01165        # orthophoto step recorded in the repo
processing_gsd_m = 0.15           # cache used so far; changing it is a recorded dec

[models]
proposer  = "sam2_hiera_tiny"     # or "roof_unet_rgb_ndsm_v1" after R7 training
refiner   = "none"                # or "sam2_1_tiny"
judge     = "embedding_probe"     # "vlm_audit" only for audits, never for recall
```

```
[policy]
roof_to_ground   = "?"            # roof_outline | ground_footprint | both
eave_offset_m    = "?"            # empty until measured
ndsm_min_height_m = 2.5           # current rule in the pipeline

[gates]
object_recall_min       = "?"     # set by you and the surveyor
boundary_distance_p90_m = "?"
invalid_geometries_max  = 0

[legal]
urbanism_basis  = "Law 169/2026 (Code)"
pug_data_schema = "Order 904/2023, GPKG, EPSG:3844"
cadastre_basis  = "ANCPI Order 600/2023"

[profiles.laptop]
gpu_vram_gb = 6                   # confirm the card; the repo machine reports an RTX
llm         = "cloud"
vlm         = "off"

[profiles.workstation]
gpu_vram_gb = 24                  # confirm the card
llm         = "local_qwen_27b_q4"
vlm         = "audit_only"
```

The run copies its own setup sheet into the output folder with the tool versions and input hashes. Any result can then be reproduced and compared, which is the property that is hardest to get from an agent conversation **[EST]**.

7. How the two machines fit in

Role	Workstation	Laptop
Build-time	Runs long agent jobs, training, local Qwen fallback, test suite on the full data	Light edits, review, QGIS checks
Run-time	Training, batch scans, SAM 3 baseline, VLM audits	Inference with the small models, QGIS client, review by the surveyor
Git	Source of the main branch for long jobs	Feature branches for review work

Both machines use the same repo, the same setup sheets (different hardware profile) and the same verify command. Do not copy trained weights or data through chat; track them by hash in the manifest **[EST]**. Confirm the exact GPU models first, as noted in the other reports.

8. Checks that tell you each step worked

Step	Pass check
R0	Tag exists; baseline test output saved with date and commit
R1	A new agent can find the engine, tests and state file from the README alone; `git ls-files` count of non-code files drops substantially (target set by you)

Step	Pass check
R2	`AGENTS.md` plus `STATE.md` are loaded by OpenCode and Antigravity (confirmed in each tool); 4 agent roles remain
R3	The entry command reproduces the Cluj figures within rounding
R4	Each of the 221 scripts has a class; promoted ones run as sub-commands
R5	For each stage, old and new outputs are identical in golden tests; no root module imports a client
R6	MCP tool list is about eight; unused providers are removed or marked excluded

9. Rules that keep everyone on the line

1. Nothing starts without a task card that names the number or gate it moves.
2. The headline metric is the object recall and boundary distance on the frozen area. If a change cannot move it, it waits.
3. New top-level documents are not allowed. Append to the task card or `STATE.md`.
4. New agents, providers or UI features need your written approval (the rule is already in `AGENTS.md`) **[REPO]**.
5. Weekly, 30 minutes: archive finished task cards, check that `STATE.md` is true, delete nothing without your approval.
6. If an agent fails the same check twice, it stops and writes the handoff; it does not try a third variation **[EST]**.
7. The LLM never approves a result and never produces a coordinate.

10. What I did not verify

- The git history, branch layout and which agent made which change (the clone had one commit).
- Whether the test suite passes today (not run).
- Whether Antigravity discovers your flat agent files, and whether the four rule files have the frontmatter Antigravity expects.
- Whether OpenCode sees any of the `.agents` content; documented behaviour covers `AGENTS.md` and `opencode.json` instructions only.
- The exact subscription limits of your plans. Check them before assigning long autonomous jobs.
- The import counts come from a static scan with regular expressions and could miss dynamic imports.

Sources

Repository files read or counted: `AGENTS.md`, `CLAUDE.md`, `GEMINI.md`, `.kilorules`, `.agents/`, `docs/PROJECT_STATE.md`, `docs/AGENT_HANDOFF.md`, `docs/SYSTEM_MAP.md`, `docs/STRATUMRO_PRODUCT_IDENTITY_AND_DESKTOP_DIRECTION_2026-10-17.md`, `docs/architecture/MIGRATION_PLAN.md`,

`docs/architecture/STRATUMRO_FINAL_ARCHITECTURE_SYNTHESIS.md`, `docs/STRATUMRO_AGENT_DECISION_PROTOCOL.md`, `docs/LESSONS_LEARNED.md`, `stratum_ro/`, `tools/`, `mcp/`, `scripts/`. Repository: StratumRO-QGIS.

External:

- OpenCode rules
- Google Antigravity rules, older rules and workflows page
- Antigravity CLI agents
- Gemini API managed agents with AGENTS.md and SKILL.md
- Anthropic, Building effective agents